PROYECCIÓN SOCIAL | ENTREGA SEMANA 4

RECICLA CERCA

EcoRedSV

Creación del proyecto en Django y configuración de SQL Server

Estructura inicial de la aplicación web, primeras clases del sistema y conexión comprobada con la base de datos.

Equipo	Grupo 6
Proyecto	Recicla Cerca
Comunidad de referencia	Colonia Veracruz, Chalatenango
Asignatura	Programación Orientada a Objetos
Fecha	Octubre de 2026
Docente / institución	Francisco Daniel Peñate Mena / Universidad Dr. Andrés Bello

INTEGRANTES

Carlos Fernando Guardado García
Liliana Dalila Ulloa González
Daniela Ester López Varela
Robin Christopher Barrera Vásquez
Alexis Omar García Romero

Contenido

1. Datos de la semana y modalidad de trabajo
2. Integrantes y responsabilidades
3. Creación y estructura del proyecto
4. Clases, funciones y módulos desarrollados
5. Configuración de SQL Server
6. Evidencias de ejecución
7. Problemas encontrados y soluciones
8. Evidencias de Git
9. Enlaces

1. Datos de la semana y modalidad de trabajo

En la Semana 4 se creó el proyecto en Django, se conectó con SQL Server y se programaron las primeras clases del sistema, basadas en el diseño de la Semana 3.

Semana	Semana 4 de 8
Tema	Creación del proyecto en Django y configuración de SQL Server
Punto de partida	Diagrama de clases, modelo de datos y requisitos actualizados en la Semana 3

Objetivo de la semana: construir la estructura inicial de EcoRedSV y comprobar que Django se ejecuta correctamente y se conecta con la base de datos.

Modalidad de trabajo: igual que en la Semana 3, cada integrante desarrolla la actividad de su rol y una sola persona integra los aportes en este documento para mantener un formato uniforme. La participación de cada integrante se evidencia en la tabla de responsabilidades, en sus comentarios de revisión dentro del documento y en sus commits en GitHub.

2. Integrantes y responsabilidades

Los roles de las semanas anteriores se mantienen, ajustados a las actividades técnicas de esta etapa.

N.º	Integrante	Rol	Actividad de la Semana 4	Evidencia
1	Carlos Fernando Guardado García	Desarrollo y base de datos	Entorno virtual, proyecto Django, apps, clases del dominio, página de inicio, configuración de SQL Server e integración del documento.	Commits del proyecto y secciones 3 a 8

N.º	Integrante	Rol	Actividad de la Semana 4	Evidencia
2	Liliana Dalila Ulloa González	Análisis y documentación	Revisión del documento de la semana y verificación de que las clases y requisitos atendidos coincidan con lo definido en la Semana 3.	Comentarios de revisión en el documento
3	Daniela Ester López Varela	Diseño e interfaz	Revisión de la página de inicio y de la estructura del proyecto, con propuestas de mejora para la interfaz de las próximas semanas.	Comentarios de revisión en el documento
4	Robin Christopher Barrera Vásquez	Pruebas y calidad	Subir el PDF de la Semana 4 a la carpeta docs del repositorio.	Commit en GitHub
5	Alexis Omar García Romero	Gestión y evidencias	Clonar el repositorio, seguir las instrucciones del README para comprobar que la instalación funciona y subir los ajustes necesarios.	Commit en GitHub

Además, cada integrante deja comentarios de retroalimentación a lo largo de este documento, en las secciones relacionadas con su rol. Esos comentarios quedan como evidencia de su revisión en Google Docs.

3. Creación y estructura del proyecto

El proyecto se creó dentro del repositorio existente de GitHub, con un entorno virtual de Python 3.12, Django 6.1 y cinco aplicaciones que corresponden a los módulos del diseño de la Semana 3.

3.1 Entorno virtual y dependencias

Los comandos se ejecutaron en Linux Mint (basado en Ubuntu 24.04), dentro de la carpeta del repositorio:

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install mssql-django python-decouple
pip freeze > requirements.txt
```

Paquete	Versión	Uso en el proyecto
Django	6.1.2	Framework principal del backend.
mssql-django	2.0.0	Conector de Django con SQL Server.
pyodbc	5.3.0	Controlador de Python que usa el ODBC Driver 18.
python-decouple	3.8	Lee las credenciales desde el archivo .env, fuera del código.

El archivo requirements.txt guarda la lista completa de dependencias con sus versiones exactas, para que cualquier integrante instale el mismo entorno con `pip install -r requirements.txt`.

3.2 Creación del proyecto y de las aplicaciones

```
django-admin startproject config .
python manage.py startapp core
python manage.py startapp usuarios
python manage.py startapp puntos
python manage.py startapp campanias
python manage.py startapp reportes
```

El punto final de `startproject config .` crea `manage.py` en la raíz del repositorio. Las cinco aplicaciones se registraron en `INSTALLED_APPS` dentro de `config/settings.py`.

Aplicación	Módulo de la Semana 3	Contenido
core	Clases base	Página de inicio, EntidadBase, Ubicable y la función del mapa.
usuarios	Usuarios	Roles, acciones y permisos.
puntos	Puntos y materiales	Material y PuntoRecepcion.
campanias	Campañas	Campania, Participacion y RegistroRecoleccion.
reportes	Reportes y participación	ReporteAmbiental, SeguimientoReporte y Comentario.

3.3 Estructura del proyecto

```
EcoRedSV/
├── config/          Configuración de Django (settings.py, urls.py)
├── core/            Página de inicio y clases base
├── usuarios/        Roles y permisos
├── puntos/          Puntos de recepción y materiales
├── campanias/       Campañas, inscripciones y recolección
├── reportes/        Reportes, seguimiento y comentarios
├── docs/            Diagramas y PDF de cada entrega
├── manage.py
├── requirements.txt
├── .env.example     Plantilla de variables de entorno
├── .gitignore
└── README.md
```

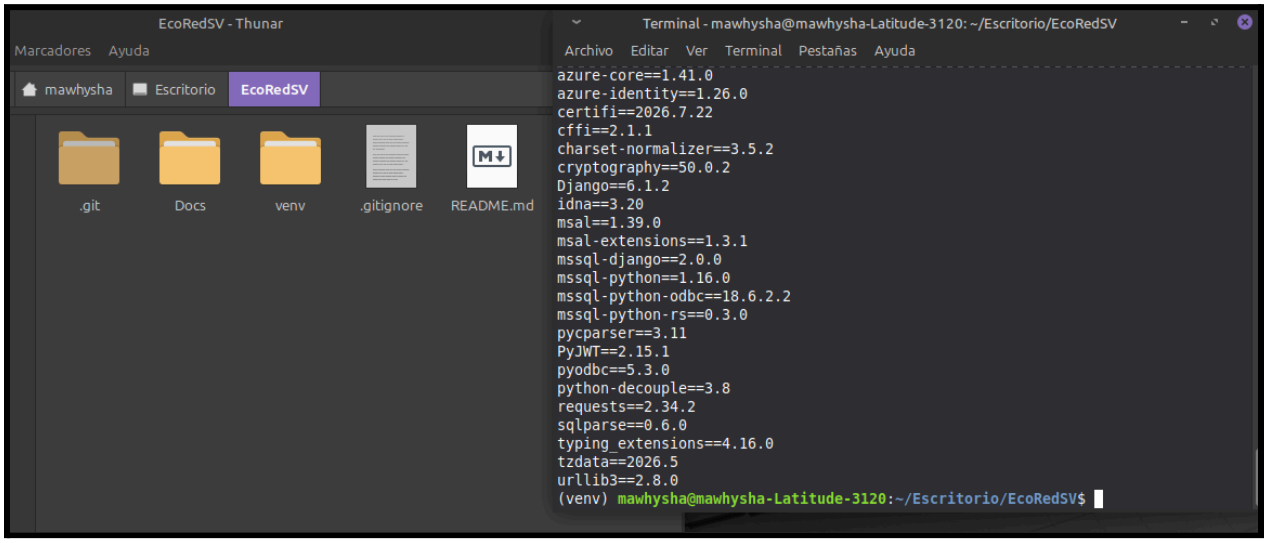
El entorno virtual (venv) y el archivo .env con las credenciales existen solo en cada computadora: el .gitignore los excluye del repositorio.

3.4 Evidencias

Evidencia 1. Entorno virtual activo

Descripción: la terminal muestra (venv) al inicio de la línea, lo que confirma que los comandos se ejecutan dentro del entorno virtual del proyecto.

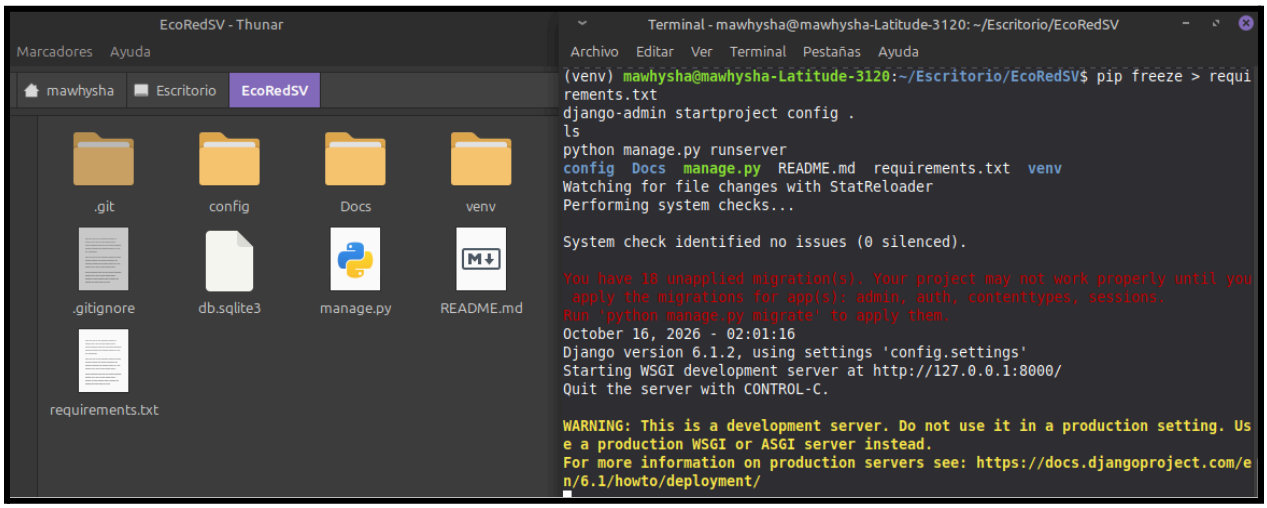
[CAPTURA 1: Entorno virtual activo]



Evidencia 2. Dependencias instaladas

Descripción: salida de pip freeze con las versiones exactas instaladas, entre ellas Django 6.1.2, mssql-django 2.0.0, pyodbc 5.3.0 y python-decouple 3.8. Es la misma lista guardada en requirements.txt.

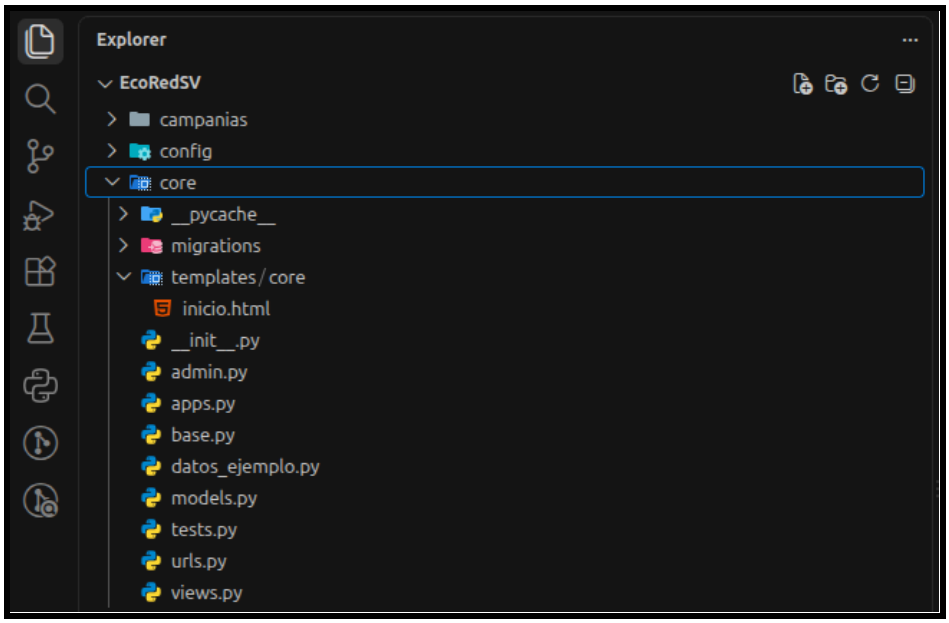
[CAPTURA 2: Dependencias instaladas]



Evidencia 3. Estructura de la aplicación core

Descripción: explorador de VS Code con la aplicación core desplegada: base.py, datos_ejemplo.py, views.py, urls.py y la plantilla inicio.html.

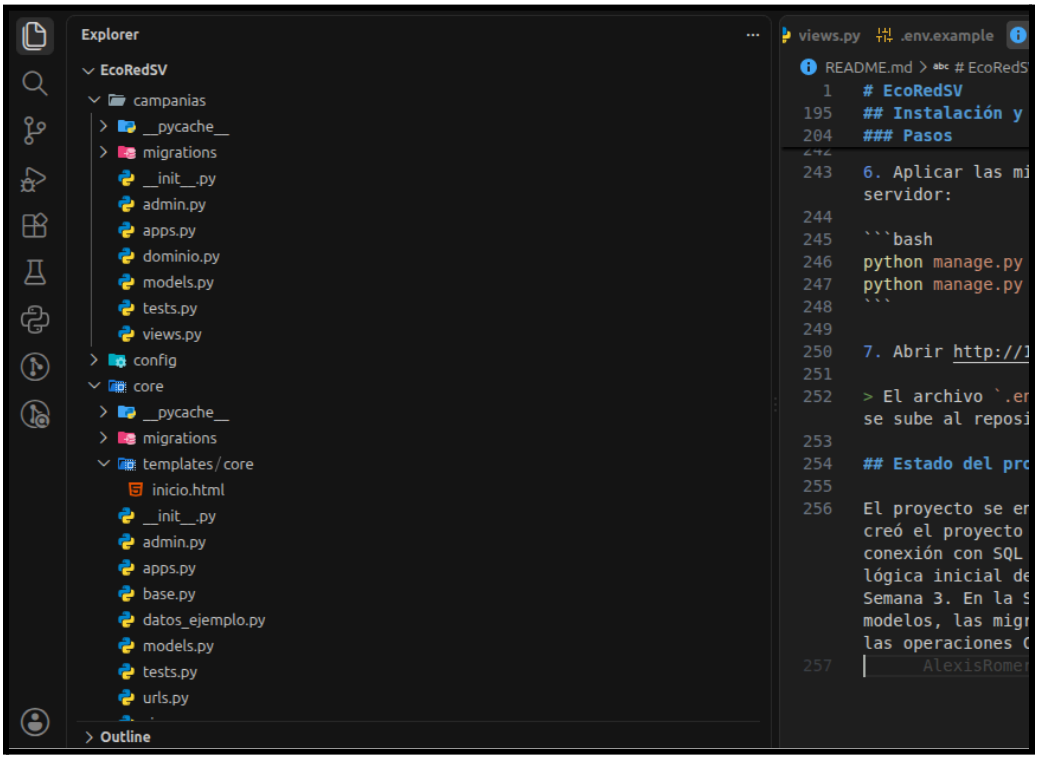
[CAPTURA 3: Estructura de la aplicación core]



Evidencia 4. Estructura de las demás aplicaciones

Descripción: aplicaciones usuarios, puntos, campanias y reportes desplegadas, con los módulos permisos.py y dominio.py de cada una.

[CAPTURA 4: Estructura de las demás aplicaciones]



4. Clases, funciones y módulos desarrollados

La lógica principal del sistema se programó en Python, en un módulo por aplicación, siguiendo las clases del diagrama de la Semana 3. En esta etapa las clases trabajan en memoria; en la Semana 5 se conectarán a los modelos de Django y a las tablas de SQL Server.

4.1 Módulos y clases

Archivo	Clases y funciones	Relación con la Semana 3
core/base.py	EntidadBase, Ubicable, construir_marcadores()	Clases abstractas del diagrama y polimorfismo del mapa.
usuarios/permisos.py	Rol, Accion, PerfilUsuario	Roles de la sección 3.4, tiene_permiso() y puede_editar().
puntos/dominio.py	Material, PuntoRecepcion, filtrar_puntos()	PuntoMaterial, estado de la información (RF-19) y filtro por municipio y material (RF-05).
campanias/dominio.py	Campania, Participacion, RegistroRecoleccion	Inscripción (RF-16), peso recolectado (RF-17) y resultados (RF-18).
reportes/dominio.py	ReporteAmbiental, SeguimientoReporte, Comentario, ordenar_por_apoyos()	Gestión de reportes (RF-11), apoyos (RF-20), comentarios (RF-21) y moderación (RF-22).
core/datos_ejemplo.py	crear_datos_ejemplo()	Crea objetos ficticios para comprobar que las clases funcionan juntas.
core/views.py	obtener_estado_bd(), inicio()	Consulta la conexión con SQL Server y arma la página de inicio.

La clase de usuarios se llama PerfilUsuario y no Usuario a propósito: en la Semana 5 el modelo de la base de datos se llamará Usuario y usará esta lógica de permisos, sin que los nombres choquen.

4.2 Aplicación de Programación Orientada a Objetos

Concepto	Cómo se aplica en el código
Clases y objetos	Cada elemento del problema es una clase; crear_datos_ejemplo() crea objetos concretos: un punto, una campaña, dos reportes y cinco usuarios.
Atributos y métodos	PuntoRecepcion guarda nombre, horario y materiales, y ofrece agregar_material(), confirmar_informacion() y mostrar_detalle().
Herencia	EntidadBase → Ubicable → PuntoRecepcion, Campania y ReporteAmbiental.
Polimorfismo	construir_marcadores() llama a obtener_marcador() sin conocer el tipo de cada elemento; cada clase responde con su propio marcador.

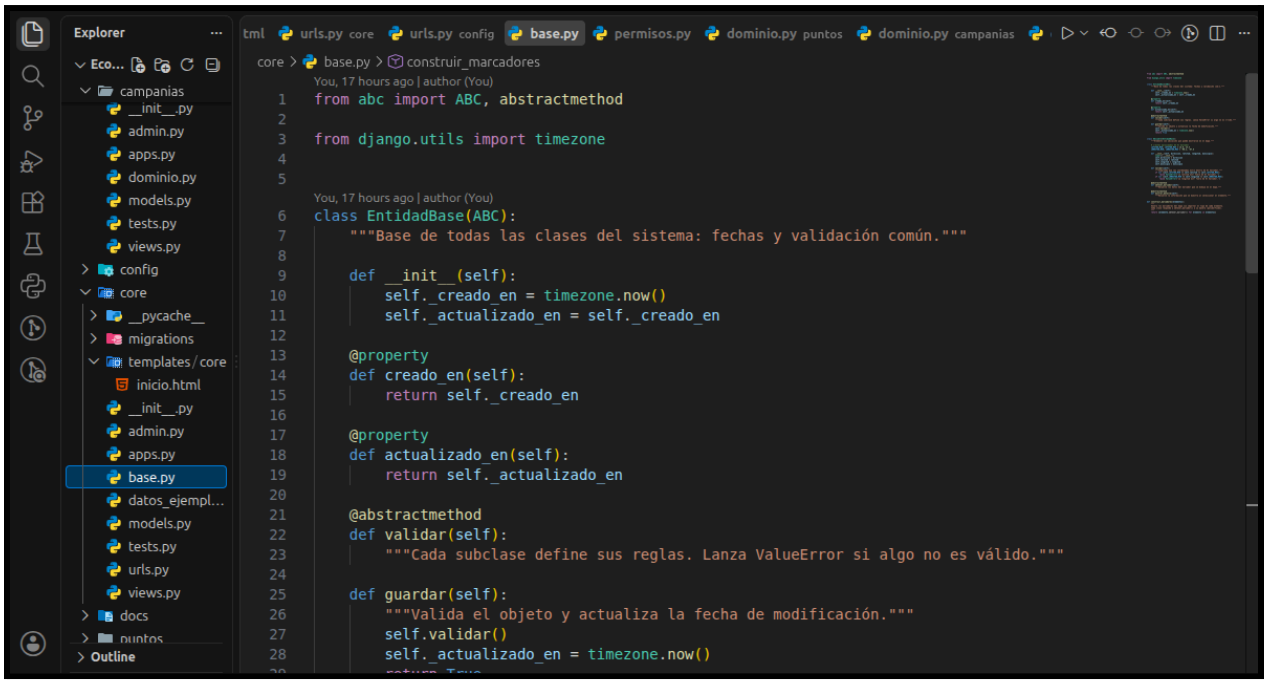
Concepto	Cómo se aplica en el código
Abstracción	EntidadBase y Ubicable usan ABC y @abstractmethod: no se instancian y obligan a las subclases a definir validar(), obtener_marcador() y mostrar_detalle().
Encapsulamiento	El peso de RegistroRecoleccion se asigna mediante un @property que rechaza valores menores o iguales a cero; los estados de campañas y reportes solo cambian mediante cambiar_estado(), que revisa las transiciones permitidas.
Funciones	filtrar_puntos(), ordenar_por_apoyos(), construir_marcadores() y obtener_estado_bd() resuelven tareas concretas fuera de las clases.
Módulos conectados	Campania y ReporteAmbiental consultan PerfilUsuario.tiene_permiso() antes de inscribir, registrar pesos, apoyar, comentar o moderar.

4.3 Evidencias

Evidencia 5. Código de core/base.py

Descripción: clases abstractas EntidadBase y Ubicable, definidas con ABC y @abstractmethod, y la función construir_marcadores(), que aplica polimorfismo.

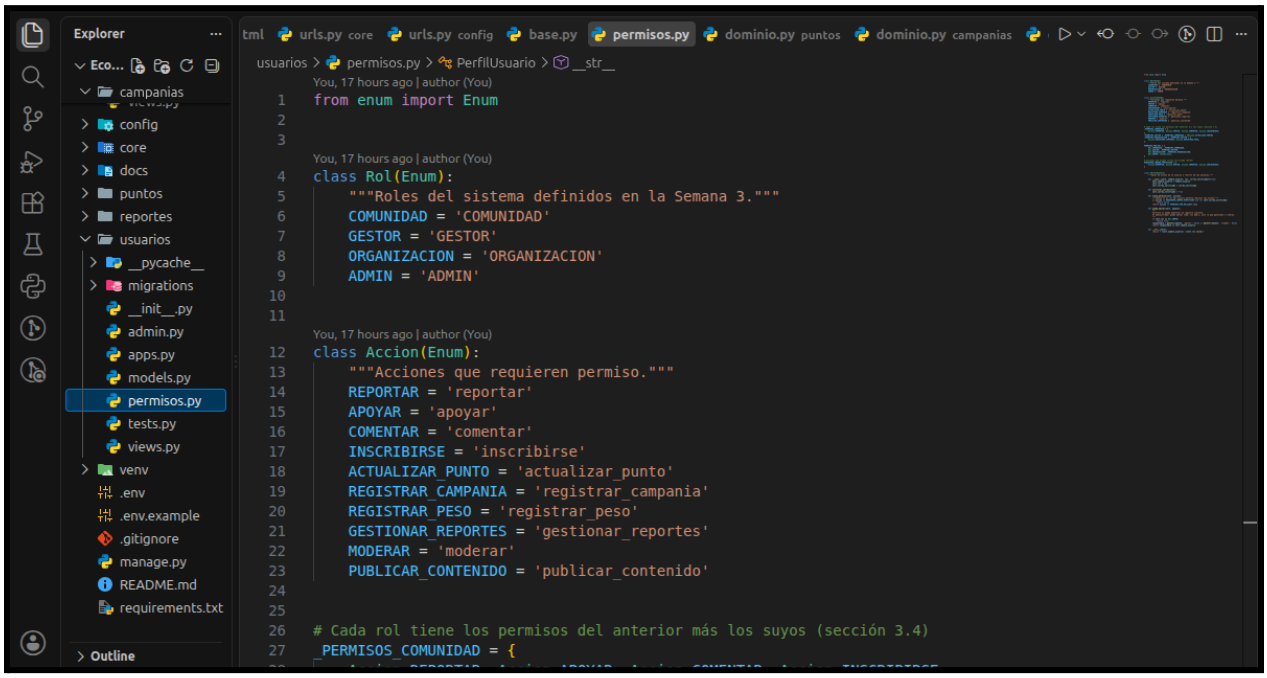
[CAPTURA 5: Código de core/base.py]



Evidencia 6. Código de usuarios/permisos.py

Descripción: enumeraciones Rol y Accion, permisos acumulados por rol y la clase PerfilUsuario con los métodos tiene_permiso() y puede_editar().

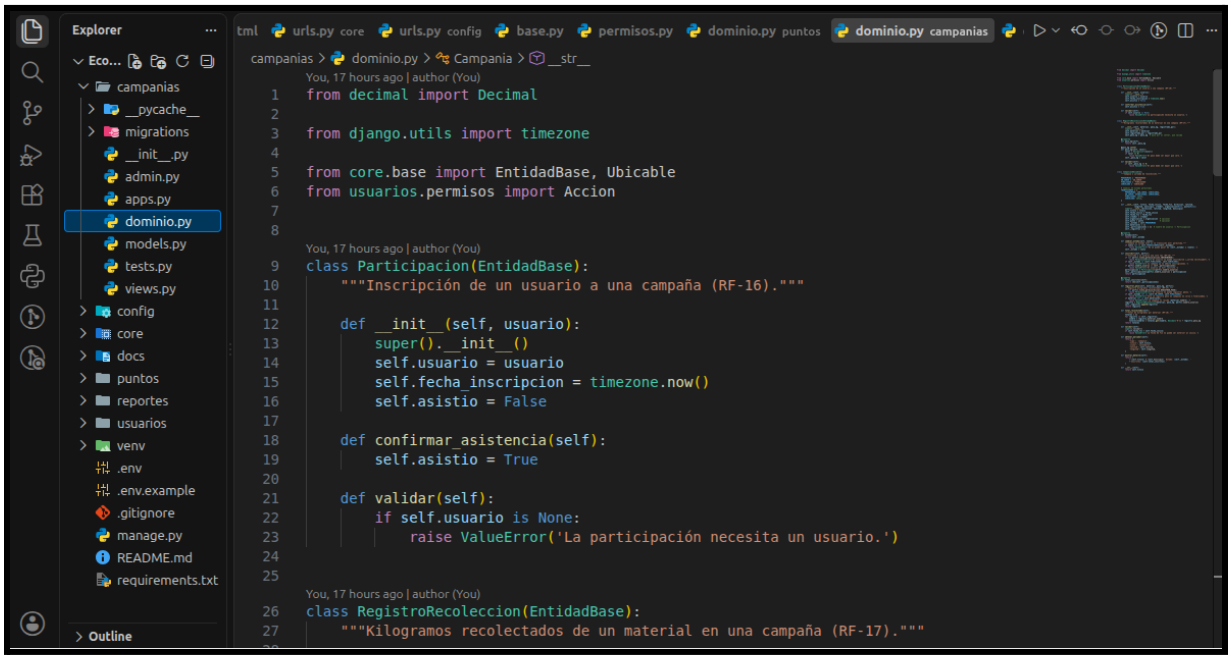
[CAPTURA 6: Código de usuarios/permisos.py]



Evidencia 7. Código de campanias/dominio.py

Descripción: RegistroRecoleccion con el peso protegido por @property, y la clase Campania, que hereda de Ubicable, valida sus cambios de estado y consulta los permisos antes de inscribir o registrar pesos. Puede ir en dos imágenes.

[CAPTURA 7: Código de campanias/dominio.py]



5. Configuración de SQL Server

Django quedó conectado a SQL Server 2025 en Linux mediante mssql-django y el ODBC Driver 18, con un usuario propio para el proyecto y las credenciales fuera del código.

Servidor	SQL Server 2025 en Linux, instalación local
Base de datos	ecoredsv
Usuario del proyecto	ecoredsv_app, con el rol db_owner solo en ecoredsv
Controlador	ODBC Driver 18 for SQL Server
Conector de Django	mssql-django 2.0.0 con pyodbc
Gestión de la base	DBeaver

5.1 Instalación del controlador ODBC

Linux Mint no es reconocido por el instalador de Microsoft, así que se usó el repositorio de Ubuntu 24.04, la versión en la que se basa el sistema:

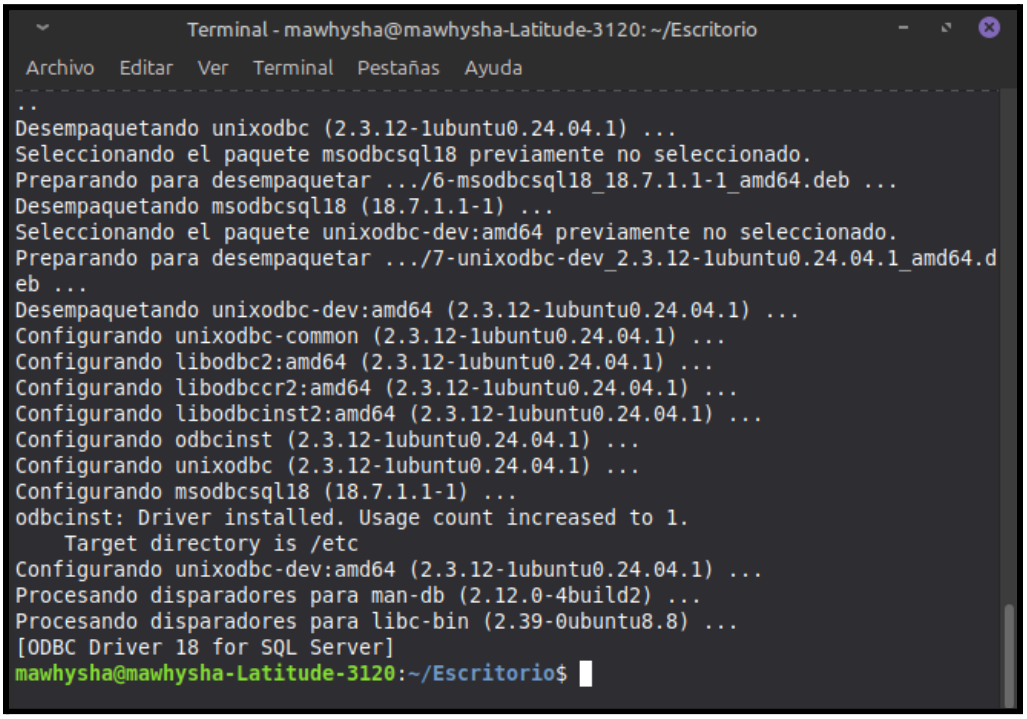
```
curl -sSL -O https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt update
sudo ACCEPT_EULA=Y apt install -y msodbcsql18 unixodbc-dev
odbcinst -q -d
```

El último comando confirmó el controlador instalado: `[ODBC Driver 18 for SQL Server]`.

Evidencia 8. Controlador ODBC instalado

Descripción: instalación de msodbcsql18 y salida de odbcinst -q -d con el controlador ODBC Driver 18 for SQL Server registrado.

[CAPTURA 8: Controlador ODBC instalado]



5.2 Base de datos y usuario

Se creó un usuario exclusivo para Django en lugar de usar la cuenta sa. Así, si la contraseña se filtrara, solo quedaría expuesta esta base de datos.

```
CREATE DATABASE ecoredsv;
CREATE LOGIN ecoredsv_app WITH PASSWORD = '********';
USE ecoredsv;
CREATE USER ecoredsv_app FOR LOGIN ecoredsv_app;
ALTER ROLE db_owner ADD MEMBER ecoredsv_app;
```

5.3 Variables de entorno

Las credenciales se guardan en un archivo .env que no se sube al repositorio. En GitHub solo se publica .env.example, con valores de ejemplo para que cada integrante cree el suyo:

```
SECRET_KEY='cambia-esta-clave'
DEBUG=True
DB_NAME=ecoredsv
DB_USER=tu_usuario
DB_PASSWORD=tu_contraseña
DB_HOST=localhost
DB_PORT=1433
```

5.4 Conexión en settings.py

`config/settings.py` lee cada dato del .env con la función config() de python-decouple. La SECRET_KEY original que genera Django también se movió al .env.

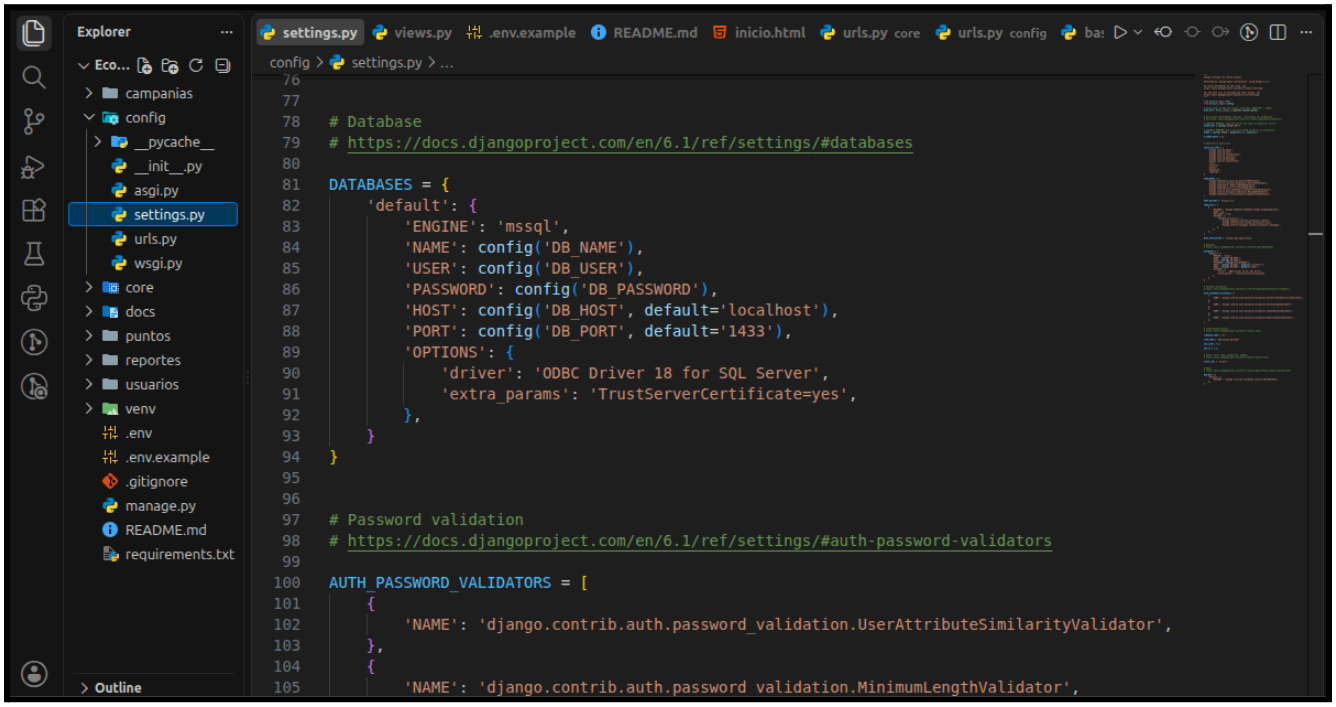
```
DATABASES = {
    'default': {
        'ENGINE': 'mssql',
        'NAME': config('DB_NAME'),
        'USER': config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST': config('DB_HOST', default='localhost'),
        'PORT': config('DB_PORT', default='1433'),
        'OPTIONS': {
            'driver': 'ODBC Driver 18 for SQL Server',
            'extra_params': 'TrustServerCertificate=yes',
        },
    }
}
```

`TrustServerCertificate=yes` es necesario porque el ODBC Driver 18 cifra la conexión por defecto y el servidor local no tiene un certificado firmado. También se configuró el idioma en español (`LANGUAGE_CODE = 'es'`) y la zona horaria de El Salvador (`TIME_ZONE = 'America/El_Salvador'`).

Evidencia 9. Configuración en settings.py

Descripción: bloque DATABASES con el motor mssql, el controlador ODBC 18 y los datos de conexión leídos desde el .env mediante config().

[CAPTURA 9: Configuración en settings.py]



5.5 Comprobación de la conexión

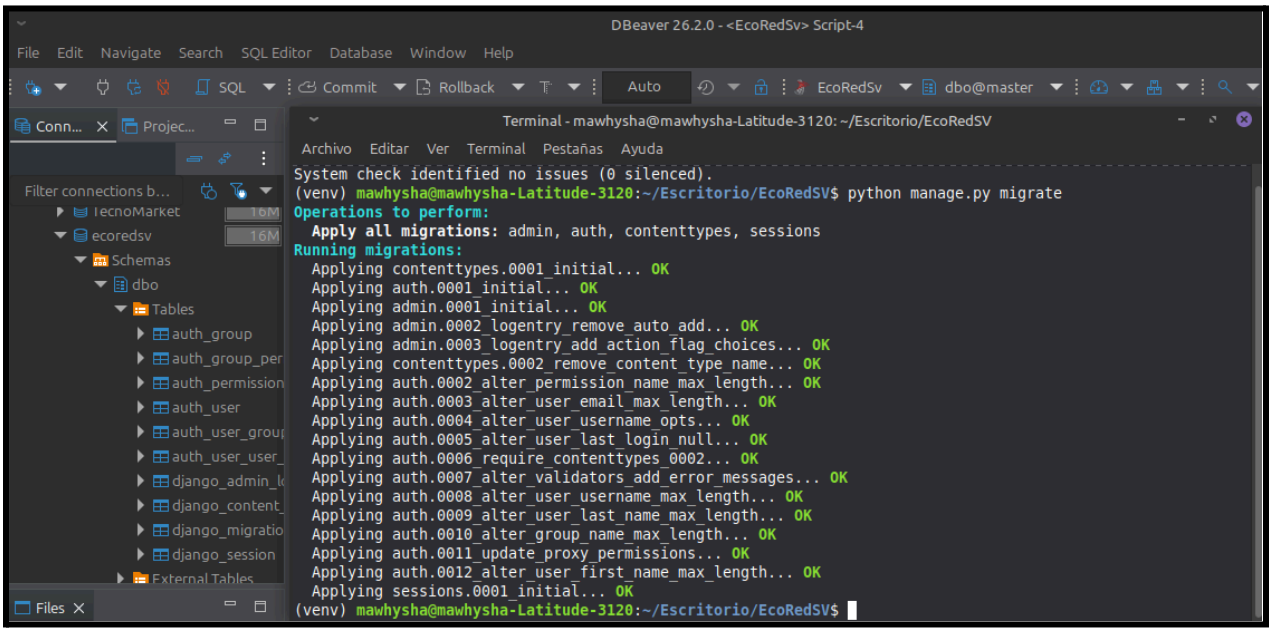
```
python manage.py check
python manage.py migrate
```

Las 18 migraciones iniciales de Django se aplicaron sin errores y sus tablas (auth_user, django_migrations, django_session y otras) aparecieron en la base ecoredsv dentro de DBeaver. Eso confirma que Django lee y escribe en SQL Server.

Evidencia 10. Migraciones aplicadas en SQL Server

Descripción: las 18 migraciones aplicadas con OK en la terminal y las tablas de Django creadas dentro de la base ecoredsv en DBeaver.

[CAPTURA 10: Migraciones aplicadas en SQL Server]



6. Evidencias de ejecución

La aplicación se ejecuta con `python manage.py runserver` y responde en http://127.0.0.1:8000 con una página de inicio propia que comprueba la conexión y las clases en el momento de cargarla.

La página tiene dos partes:

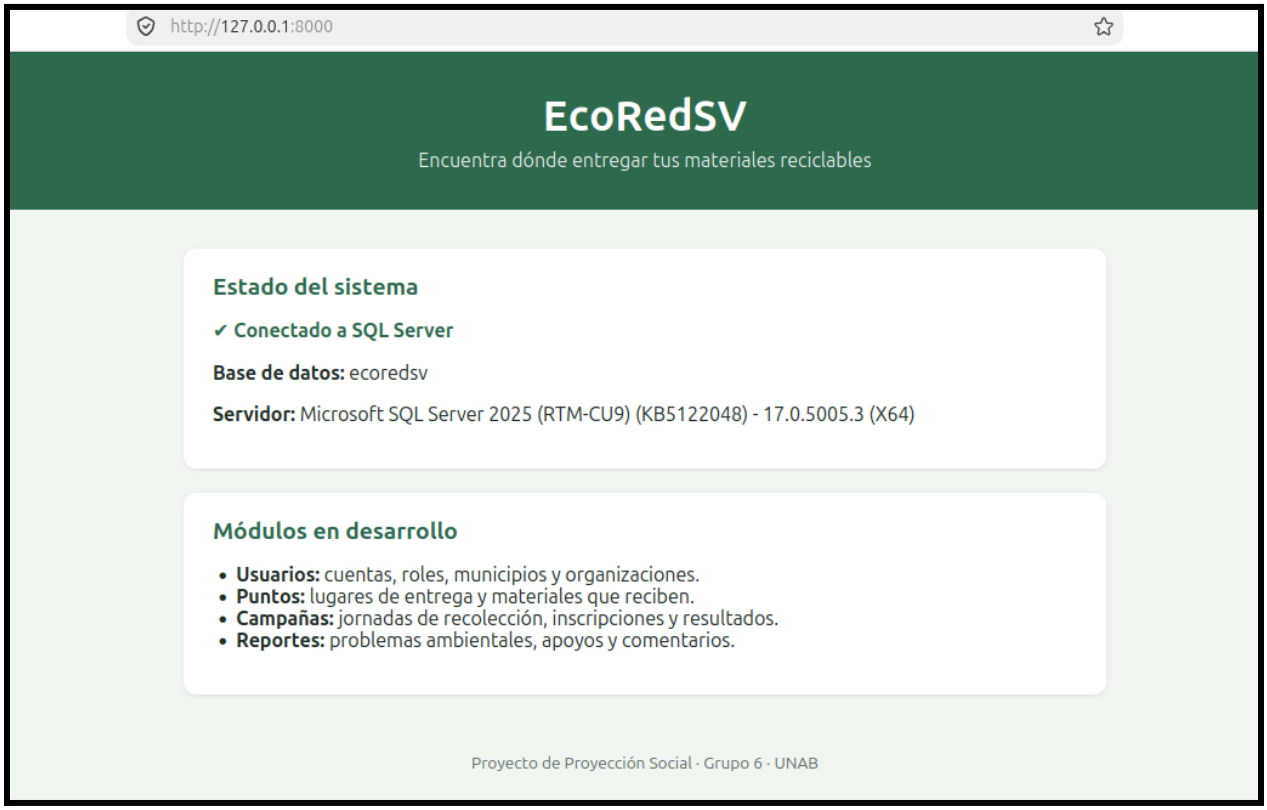
- **Estado del sistema:** consulta en vivo `SELECT DB_NAME(), @@VERSION` y muestra la base de datos (ecoredsv) y la versión de SQL Server.
- **Prueba de clases:** crea objetos de ejemplo y muestra los marcadores del mapa, el detalle de cada elemento, los kilogramos recolectados por material, los reportes ordenados por apoyos, el historial de estados, los permisos por rol y un peso negativo rechazado por la validación.

Los datos de la prueba son ficticios y existen solo en memoria; no se guardan en la base de datos.

Evidencia 11. Página de inicio y conexión

Descripción: página en http://127.0.0.1:8000 con el estado "Conectado a SQL Server", la base ecoredsv y la versión del servidor.

[CAPTURA 11: Página de inicio y conexión]



Evidencia 12. Prueba de clases

Descripción: resultados de los objetos de ejemplo: marcadores del mapa, detalle de cada elemento, kilogramos recolectados, reportes ordenados por apoyos, historial de estados, permisos por rol y la validación del peso negativo.

[CAPTURA 12: Prueba de clases]



7. Problemas encontrados y soluciones

Se registraron cinco problemas durante la configuración; todos se resolvieron y ninguno requirió cambiar el diseño de la Semana 3.

N.º	Problema	Causa	Solución
1	El comando odbcinst no existía y el sistema no tenía controlador ODBC para SQL Server.	DBeaver se conectaba sin problema, pero usa JDBC (Java), no ODBC. Django sí necesita el controlador ODBC de Microsoft.	Se instaló msodbcsql18 desde el repositorio de Microsoft para Ubuntu 24.04, la versión base de Linux Mint, y se comprobó con odbcinst -q -d.
2	Error 15023 al crear el usuario: el usuario ecoredsv_app ya existía en la base de datos.	El script se ejecutó más de una vez y el usuario ya se había creado en un intento anterior.	Se volvió a asignar el rol con ALTER ROLE y una consulta confirmó que el usuario existía solo en ecoredsv con el rol db_owner.
3	VS Code marcaba el aviso "Import decouple could not be resolved".	El editor usaba el Python del sistema y no el del entorno virtual, donde estaba instalado el paquete.	Se abrió la carpeta completa del proyecto con code . y se seleccionó ./venv/bin/python como intérprete.
4	Apareció un archivo db.sqlite3 en el repositorio.	La primera ejecución del servidor ocurrió antes de configurar SQL Server, cuando Django todavía usaba SQLite por defecto.	Se eliminó el archivo, ya que el proyecto trabaja únicamente con SQL Server.

N.º	Problema	Causa	Solución
5	La página mostraba el texto {{ cambio.comentario }} en lugar del comentario del historial.	El formateador de VS Code partió la etiqueta de la plantilla en dos líneas al guardar, y Django no reconoce etiquetas partidas.	Se reorganizó el bloque en líneas cortas para que el formateador no las divida.

Pendiente para la Semana 5: las migraciones iniciales crearon la tabla de usuarios por defecto de Django (auth_user). Como el diseño usa una clase Usuario propia con rol y correo verificado, la base se reiniciará antes de crear los modelos. No se pierde información, porque todavía no hay datos reales.

8. Evidencias de Git

El avance de la semana se subió al repositorio en commits separados por tema.

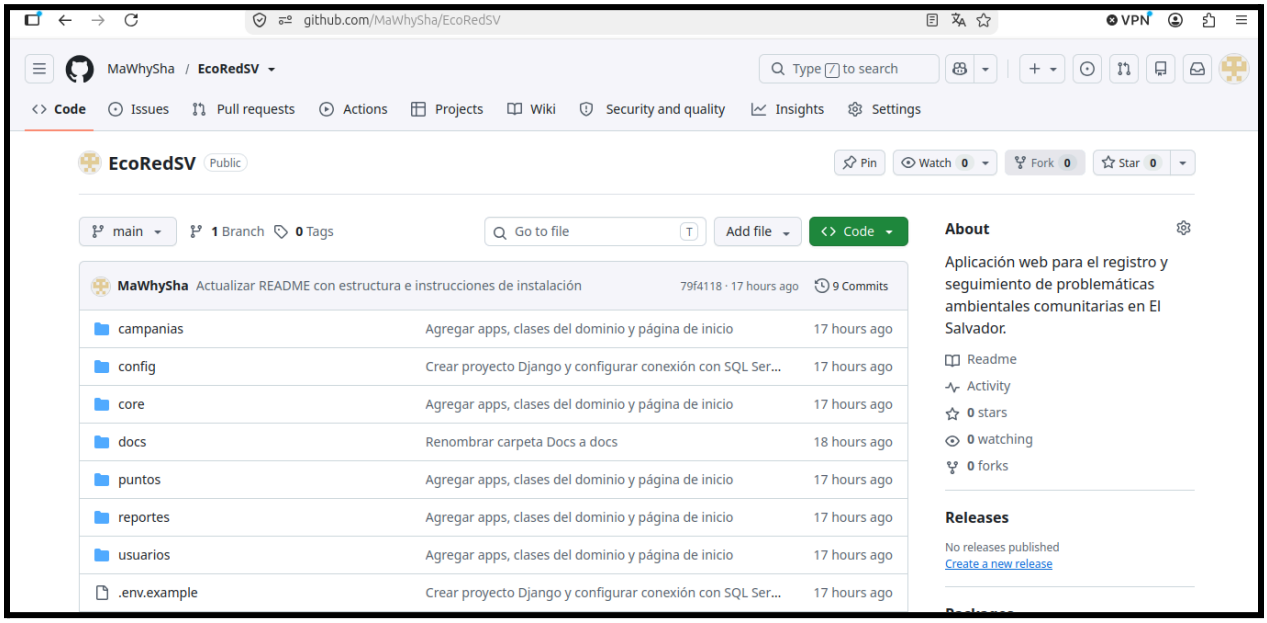
Commit	Descripción
79f4118	Actualizar README con estructura e instrucciones de instalación
478958d	Agregar apps, clases del dominio y página de inicio
428c946	Crear proyecto Django y configurar conexión con SQL Server
cecbcce	Renombrar carpeta Docs a docs

El repositorio contiene la estructura del proyecto, requirements.txt, .gitignore, .env.example y el README con las instrucciones de instalación y ejecución. No incluye el entorno virtual, el archivo .env, contraseñas, archivos temporales ni carpetas generadas automáticamente.

Evidencia 13. Repositorio en GitHub

Descripción: página principal del repositorio con las carpetas del proyecto, requirements.txt, .env.example y el README actualizado.

[CAPTURA 13: Repositorio en GitHub]



Evidencia 14. Historial de commits

Descripción: historial de commits en GitHub con los cambios respectivos de la Semana 4.

9. Enlaces

Recurso	Enlace	Permisos
Google Docs principal	https://docs.google.com/document/d/1aSZ90jYoQQPBdBpHOCxw4lpaN_MYYtPjztaSrMe9FOA/edit?usp=drive_link	Visualización habilitada para el docente
Repositorio	https://github.com/MaWhySha/EcoRedSV.git	Visualización habilitada para quien tenga el enlace